

Rapport Technique : UtilityHub - Système Modulaire de Gestion d'Outils Utilitaires

Avertissement : Ce projet a été mené strictement à des fins pédagogiques pour étudier les protocoles de streaming vidéo, l'extraction de données et les techniques d'automatisation. Aucun contenu protégé par des droits d'auteur n'a été distribué ou utilisé au-delà de ce qui est légalement autorisé. Toutes les expériences ont été limitées à des données accessibles au public.

Note pour l'Évaluateur : Priorité de l'Interface Desktop

Bien que le projet propose une architecture hybride (Web + Desktop), l'application **Desktop (PyQt5)** a bénéficié de la majeure partie du temps de développement et constitue la version la plus aboutie, performante et complète de UtilityHub.

En raison de cet investissement temporel plus important sur le client lourd, elle est plus esthétique et intègre des **fonctionnalités supérieures** (telles que l'annulation de tâches en temps réel ou une ergonomie de multithreading avancée) qui ne sont pas encore totalement déclinées sur la version Web.

Il est donc fortement recommandé d'utiliser la version Desktop pour évaluer la pleine mesure du travail technique réalisé. Une démonstration vidéo ([presentation_video_pyqt.mp4](#)) est disponible à la racine du projet pour illustrer ces capacités.

I. Architecture Complète et Structure du Projet

L'organisation des fichiers de **UtilityHub** suit une logique de séparation stricte entre le cœur de l'application, les modules métier et les interfaces utilisateurs. Cette structure garantit une maintenance aisée et une extensibilité fluide.

1.1. Arborescence Technique du Projet

```
utilityhub/
├── app.py                # launcher web
├── desktop_app.py        # launcher desktop
├── assets/               # Ressources
│   ├── active_tab_btns/
│   ├── inactive_tab_btns/
│   ├── red_X_icon.png
│   └── blue_download_icon.png
├── tools/
│   ├── ffmpeg.exe        # Binaire FFmpeg
│   └── node.exe           # Binaire Node
├── data/
│   ├── notes.csv
│   ├── todos.csv
│   ├── history.csv
│   └── settings.json
```

```

├── uploads/           # Stockage temporaire
├── outputs/           # Fichiers générés
│   ├── converted/
│   └── downloads/
├── core/              # Moteurs de l'application
│   ├── csv_manager.py
│   ├── history_manager.py
│   └── settings_manager.py
├── modules/
│   ├── converter/
│   │   ├── converter_service.py
│   │   └── converter_routes.py
│   ├── downloader/
│   │   ├── downloader_service.py
│   │   └── downloader_routes.py
│   ├── notes/
│   │   ├── notes_service.py
│   │   └── notes_routes.py
│   └── todo/
│       ├── todo_service.py
│       └── todo_routes.py
├── web/               # Interface Flask
│   ├── templates/
│   └── static/
└── desktop/           # Interface PyQt5
    ├── main_window.py # Fenêtre principale
    └── windows/        # Fenêtres modulaires
        ├── converter_window.py
        ├── downloader_window.py
        ├── notes_window.py
        ├── todo_window.py
        └── history_window.py

```

1.2. Flux de Communication

Les interfaces (Desktop/Web) ne communiquent jamais directement entre elles. Elles partagent une instance des **Services** situés dans `modules/`, qui eux-mêmes s'appuient sur les **Managers** de `core/` pour accéder aux données sécurisées par `FileLock`.

II. Architecture Système et Gestion de la Persistance

2.1. Navigation et Menu Principal

L'application utilise un **tableau de bord central** qui gère l'ensemble du menu. La navigation a été optimisée pour permettre de passer d'un outil (Convertisseur, Téléchargeur, etc.) à l'autre sans jamais fermer ou recharger l'application. Cette approche permet de conserver le travail en cours (états des champs, logs) même lors d'un changement d'onglet.

2.2. Rapidité et Travail en Arrière-plan

Pour éviter que l'interface ne se fige (freeze) lors de tâches prolongées, nous exploitons le multithreading :

- **Tâches de fond** : Les opérations lourdes sont déléguées à des threads secondaires pour maintenir une interface fluide et réactive.
- **Signaux en temps réel** : L'outil communique sa progression (pourcentage, vitesse, ETA) via des signaux, permettant une mise à jour instantanée des barres de progression sans perturber le thread principal.

2.2. Architecture en Couches

Le système repose sur une séparation nette des responsabilités, organisée en cinq couches :

1. **Couche de Persistance** : Gestion des fichiers `notes.csv`, `todos.csv`, `history.csv` (Données) et `settings.json` (Configurations).
2. **Couche Core (Managers)** :
 - **CsvManager** : `load_csv()` avec gestion des types de colonnes, `save_csv()`, `lock()` (implémentation FileLock), et `generate_id()` (détermination de l'ID maximal + 1).
 - **HistoryManager** : `add_record()`, `get_history()` avec filtrage par outil, `delete_record()` et `clear_history()`.
 - **SettingsManager** : `get_setting()`, `update_setting()` (sauvegarde immédiate) et `get_all_settings()`.
3. **Couche Service** : Logique métier encapsulée dans des classes dédiées, isolée de toute interface graphique.
4. **Couche d'Outils Externes** : Pilotage de binaires (`FFmpeg`, `node.exe`, `vtracer.exe`) via le module `subprocess`.
5. **Couche de Présentation** : Double interface (Desktop PyQt5 et Web Flask via Blueprints).

2.4. Synchronisation et Sécurité via FileLock

Le système **FileLock** agit comme le pont de sécurité entre l'application Desktop et le site Web. Comme les deux versions partagent les mêmes fichiers CSV (notes, tâches, historique), il existait un risque de corruption de données en cas d'écriture simultanée. Le FileLock pose un verrou physique : dès qu'une interface commence à écrire, l'autre attend son tour, garantissant l'intégrité totale des données.

III. Services Fonctionnels et Intégrations Spécialisées

3.1. Le Service de Conversion (ConverterService)

Ce module centralise la transformation de données hétérogènes. Fonctions clés :

- **Documents** : `convert_md_to_pdf()` et `convert_txt_to_pdf()` utilisant `markdown`, `xhtml2pdf` (pisa) avec injections de styles CSS personnalisés pour un rendu professionnel.
- **Images** : `convert_to_jpg()` et `convert_to_png()` via `Pillow` (gestion des transparences RGBA vers RGB). Vectorisation via `convert_to_svg()` (moteur `vtracer`).
- **Vidéo/Audio** : `convert_mp4_to_mp3()` pilotant `FFmpeg` avec les paramètres `-i`, `-q:a 0` (qualité maximale), et `-map a`.

3.2. Le Service de Téléchargement (MediaDownloaderService)

Orchestration de `yt-dlp` pour le streaming et l'extraction :

- `get_media_info()` : Extraction de JSON (titre, durée, miniature, uploader) et filtrage des formats via `get_available_formats()`.
- `download_media()` : Gestion de la file d'attente et utilisation d'un `progress_hook` pour récupérer en temps réel le pourcentage, la vitesse et l'ETA.
- **Flexibilité** : Support du téléchargement séparé Audio/Vidéo ou combiné avec fusion automatique via FFmpeg.

3.3. Gestionnaires de Productivité (Notes & Todo)

- **NotesService** : `save_note()` (mode création ou édition), `list_notes()`, `search_notes()` (recherche textuelle), `search_by_tag()` (filtrage par catégorie).
- **TodoService** : `add_task()`, `delete_task()`, `change_status()` (basculement entre Status Pending/Done) et `get_tasks()` (tri automatique par date de création).

IV. Orchestration des Interfaces et Défis Techniques

4.1. Client Desktop : Ingénierie UI (PyQt5)

- **Multithreading** : Utilisation de `QThread` pour isoler les services lourds. Les signaux (`pyqtSignal`) permettent de mettre à jour les éléments de l'UI (ProgressBar, Logs) sans bloquer la boucle d'événements principale.
- **Navigation** : Interface centralisée (`main_window.py`) gérant des sous-fenêtres modulaires (`converter_window.py`, etc.) via un système d'onglets personnalisés.

4.2. Interface Web : Flask, SSE et Blueprints

- **Organisation par Modules (Blueprints)** : Le site est découpé en morceaux indépendants ("Blueprints"), fonctionnant comme de mini-applications collaboratives. Cela assure une maintenance simplifiée et une évolution granulaire du code.
- **Suivi en direct via SSE (Server-Sent Events)** : Normalement, un site Web attend la fin d'une tâche pour afficher un résultat. Pour contourner cette limite, nous utilisons le SSE.
 - **Détails techniques** : Le serveur maintient une connexion HTTP persistante ouverte. Il transmet des événements en texte brut (`data: ... \n\n`) via l'en-tête `Content-Type: text/event-stream`.
 - **Avantages** : Plus simple que les WebSockets, il gère la reconnexion automatique et traverse facilement les pare-feu via les ports HTTP standards. Dans UtilityHub, il diffuse la progression du téléchargement en temps réel vers l'interface web via l'API `EventSource`.

4.3. Défis Techniques : Le Cas du GIL

Un défi majeur a été la gestion du **Global Interpreter Lock (GIL)** de Python :

- **Téléchargement** : Essentiellement lié à l'attente réseau (I/O-bound), l'interface reste très fluide.
- **Conversion** : Étant une tâche intensive en CPU, le GIL a tendance à bloquer l'interface malgré l'usage de FFmpeg en processus externe. L'implémentation de `asyncio` aurait été la solution idéale pour

neutraliser ces micro-blocages, mais le temps de développement a limité cette intégration à la version actuelle.

- **Expérience Utilisateur** : Design sombre (Dark Mode) premium harmonisé entre PyQt5 (via feuilles de style QSS) et Web (via Bootstrap et CSS personnalisé).

4.4. Conclusion

UtilityHub représente une solution d'ingénierie complète, prouvant qu'une base de données CSV gérée proprement avec Pandas peut remplacer un SQL pour des outils desktop légers, tout en offrant une flexibilité multi-plateforme.

Rappel final : Pour une expérience exhaustive des fonctionnalités de pointe (gestion des threads, annulation de processus, UX avancée), la version **Desktop** reste la référence de ce projet.